


# UNIVERSIDAD PRIVADA DE TACNA

## FACULTAD DE INGENIERÍA

## Escuela Profesional de Ingeniería de Sistemas

**"Plataforma web para la comercialización y revalorización de productos perecibles orientada al consumo sostenible en la provincia de Tacna"**

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

***Renzo Fernando Loyola Vilca*** ***(2021072615)***

***Brant Antony Chata Choque*** ***(2020067577)***

**Tacna – Perú**

***2026***

# 1. INTRODUCCIÓN Y METODOLOGÍA

El presente informe analiza la trazabilidad del Sprint 4 de ÓrbiKa, denominado “Compra con stock consistente”, planificado del 26 al 31 de octubre de 2026, con una capacidad estimada de 8 puntos de historia. Su objetivo es confirmar un pedido sin vender más unidades de las disponibles, es decir, mantener el stock siempre consistente frente a compras válidas, inválidas y concurrentes.

La revisión relaciona el requisito comprometido con la especificación de requisitos (SRS), la arquitectura descrita (SAD), el código real del repositorio y las pruebas automatizadas que pudieron ejecutarse. Se evalúa RF-07 (comprar/reservar unidades disponibles).

*Metodología: se clonó el repositorio público del equipo, se leyó el código fuente de los módulos de pedidos (servicio, validación, controlador, rutas) y la función de base de datos que ejecuta la compra, y se ejecutaron realmente las suites de pruebas automatizadas del backend (Vitest + PGlite, una base PostgreSQL real en memoria) y del frontend (Vitest). Los resultados reportados en este documento son los que esas ejecuciones arrojaron, no estimaciones.*

*Limitación metodológica: el repositorio clonado contiene un único commit (“Implementacion Sprint 1”, 19/10/2026) que ya incluye el código de varios sprints posteriores, incluido RF-07. No existe un historial incremental que permita verificar en qué fecha exacta se construyó cada parte; por ello este informe evalúa el estado actual del código frente al compromiso del Sprint 4, no la progresión cronológica.*

# 2. ALCANCE Y COMPROMISO DEL SPRINT 4

| Campo | Definición según la planificación |
|---|---|
| Nombre | Sprint 4 — Compra con stock consistente |
| Periodo | 26–31 de octubre de 2026 |
| Capacidad | 8 puntos de historia |
| Objetivo | Confirmar un pedido sin vender más unidades de las disponibles. |
| Requisito comprometido | RF-07 |
| Dependencia | Depende de los Sprints 1–3; requiere producto publicado y disponible. |
| Entregable | Flujo de compra/reserva con pedido persistido, stock consistente y estados iniciales verificables. |
| Criterio de cumplimiento | Un pedido válido descuenta el stock exactamente una vez; una compra por encima del stock disponible falla; dos solicitudes simultáneas sobre las mismas unidades no dejan el stock en negativo. |
| Evidencia planificada | Caso de prueba, respuesta de la API y valores de stock antes/después. |

### Resumen de estado basado en la evidencia disponible

| Requisito | Evidencia en el código/documentación | Estado de trazabilidad |
|---|---|---|
| RF-07 — Compra con stock consistente | crear_pedido_transaccion() bloquea la fila del producto (FOR UPDATE), valida stock, modalidad y vencimiento, descuenta stock y crea pedido+detalle en una sola transacción, con clave de idempotencia para reintentos. | Implementado y parcialmente probado con la base de datos real (PGlite); dos de ocho casos propuestos quedan pendientes de ejecución (ver sección 4). |

# 3. REQUERIMIENTO — RF-07: COMPRA CON STOCK CONSISTENTE

| Elemento de trazabilidad | Análisis |
|---|---|
| SRS — definición | “Como comprador, quiero reservar/comprar unidades disponibles para asegurar mi pedido.” RF-07 / CU-05. Criterio de aceptación de la planificación: la cantidad no supera el stock; el pedido y la reducción de stock se confirman juntos; no hay stock negativo ante solicitudes simultáneas. |
| Caso de uso / SAD | La matriz de planificación vincula RF-07 con SRS RF-07 / CU-05 y con el módulo Pedidos / ServicioPedidos. Código base identificado: pedido.service.js y el RPC crear_pedido_transaccion. |
| Evidencia backend — servicio | backend/src/services/pedido.service.js → crearPedido(actor, input): exige rol "comprador", valida los datos de entrada con validarCompra(), rechaza cuentas inhabilitadas (actor.activo === false) y delega la operación completa a la función de base de datos vía db.rpc("crear_pedido_transaccion", {...}), incluida la clave de idempotencia. |
| Evidencia backend — base de datos | supabase/migrations/003_srs_integrity_and_images.sql define crear_pedido_transaccion(): toma un bloqueo de fila sobre el producto (SELECT ... FOR UPDATE), valida stock suficiente, estado de publicación, categoría activa, vendedor activo y vencimiento; calcula el monto con el precio vigente; inserta el pedido y su detalle; descuenta el stock; todo dentro de una única función transaccional de PostgreSQL. Además usa un bloqueo consultivo (pg_advisory_xact_lock) sobre comprador+clave de idempotencia para serializar reintentos de una misma confirmación. |
| Evidencia frontend | ProductDetail.jsx limita la cantidad seleccionable al stock disponible (input con max={producto.stock} y el ayudante cantidadValida), genera una clave de idempotencia por intento de compra con crypto.randomUUID() y la reenvía sin cambiar si el usuario reintenta la misma confirmación; marketplace.comprarProducto() envía esos datos a POST /pedidos. |
| Pruebas reales ejecutadas | Backend (PGlite, base PostgreSQL real en memoria, test/sql/database.test.js): compra válida descuenta stock una vez — PASS; confirmación repetida con la misma clave no duplica el pedido ni el descuento — PASS; reintento con datos distintos bajo la misma clave es rechazado — PASS; cuenta suspendida no puede comprar — PASS; categoría inactiva no puede comprarse — PASS. Backend (mocks, pedido.service.test.js): 19/19 pruebas PASS sobre el comportamiento del servicio. Frontend: cantidadValida(4, 3) = false — PASS. |
| Lo que NO está probado con evidencia real | Dos partes del criterio de aceptación no cuentan con un caso de prueba ejecutado contra la base de datos real: (1) una compra que exceda el stock disponible, y (2) dos solicitudes simultáneas sobre las mismas unidades. |

| Elemento de trazabilidad | Análisis |
|---|---|
| | Ambas están respaldadas únicamente por el diseño del código (el bloqueo FOR UPDATE), no por una ejecución registrada. |
| Criterio de aceptación | La cantidad no debe superar el stock; el pedido y la reducción de stock deben confirmarse juntos; dos solicitudes simultáneas no deben dejar stock negativo. Debe documentarse mediante casos ejecutados y capturas. |

***Estado de desarrollo: implementado; aceptación mayormente verificada con pruebas reales***

*El código implementa correctamente el mecanismo central del requisito: bloqueo de fila, validación de stock y disponibilidad, transacción atómica y protección contra reintentos duplicados mediante una clave de idempotencia. Cinco de las siete pruebas relevantes se ejecutaron contra una base de datos PostgreSQL real (PGlite) y pasaron. Antes de declarar RF-07 completamente aceptado conviene añadir los dos casos de prueba pendientes: compra que excede el stock y compra concurrente sobre las mismas unidades.*

## 4. MATRIZ DE CASOS DE PRUEBA

Los siguientes casos derivan del criterio de cumplimiento de la planificación. A diferencia del informe de Sprint 3, seis de los ocho casos ya cuentan con una ejecución real documentada (backend con PGlite, una base PostgreSQL real en memoria, y frontend con Vitest); los dos restantes quedan señalados como pendientes.

| ID | Requisito | Prueba y resultado | Resultado real |
|---|---|---|---|
| **CP-S04-01** | RF-07 | Comprar una cantidad disponible crea el pedido y reduce el stock exactamente una vez. | **PASS — evidencia real** |
| **CP-S04-02** | RF-07 | Una confirmación repetida con la misma clave de idempotencia no genera un segundo pedido ni descuenta el stock dos veces. | **PASS — evidencia real** |
| **CP-S04-03** | RF-07 | Un reintento que cambia los datos de la compra (p. ej. otra cantidad) bajo la misma clave es rechazado. | **PASS — evidencia real** |
| **CP-S04-04** | RF-07 | Una compra con cantidad mayor al stock disponible es rechazada y el stock permanece sin cambios. | **Pendiente de ejecución** |
| **CP-S04-05** | RF-07 | Dos solicitudes de compra simultáneas sobre las mismas unidades no dejan el stock en negativo. | **Pendiente de ejecución** |
| **CP-S04-06** | RF-07 | Una cuenta compradora suspendida (activo = false) no puede crear pedidos. | **PASS — evidencia real** |
| **CP-S04-07** | RF-07 | No se puede comprar un producto cuya categoría está inactiva. | **PASS — evidencia real** |
| **CP-S04-08** | RF-07 | La interfaz limita la cantidad seleccionable al stock disponible antes de enviar la compra. | **PASS — evidencia real** |

### Resultados reales de la ejecución de pruebas (31/10/2026)

| Suite | Archivos | Resultado |
|---|---|---|
| Backend — completo (con variables de entorno configuradas) | 10 archivos de prueba | 91 de 91 pruebas superadas |
| Backend — sin variables de entorno (.env) | 7 de 10 archivos cargan | 59 de 59 pruebas superadas; 3 archivos no cargan (incluye pedido.service.test.js) |
| Backend — pedido.service.test.js (aislado, con variables dummy) | 1 archivo | 19 de 19 pruebas superadas |
| Frontend — completo | 4 archivos de prueba | 49 de 49 pruebas superadas |

*Nota: sin un archivo backend/.env configurado, 3 de las 10 suites del backend (incluida la del servicio de pedidos) no cargan porque el cliente de Supabase exige SUPABASE_URL y SUPABASE_SERVICE_ROLE_KEY al importarse. Para esta verificación se definieron variables de entorno de prueba (no reales) solo para permitir que esas suites se ejecuten; sus 19 pruebas usan un cliente de base de datos simulado (mock) y no dependen de credenciales reales.*

## 5. VERIFICACIÓN DEL DEFINITION OF DONE

| Criterio | Estado | Evidencia / observación |
|---|---|---|
| Criterios de aceptación aceptados | Parcialmente | Compra válida, descuento único y rechazo por cuenta/categoría inhabilitada verificados; rechazo por stock insuficiente y por concurrencia aún no. |
| Pruebas y resultado registrados | Sí | 140 pruebas reales ejecutadas en total (91 backend + 49 frontend); resultados documentados en la sección 4. |
| Cambios integrados | Sí | Código de pedidos y su función SQL están en la rama principal del repositorio. |
| Sin defectos críticos abiertos | No verificable | El repositorio no expone un registro de incidencias. |
| Evidencia guardada sin credenciales ni datos reales | Sí | Resultados de prueba y fragmentos de código citados en este informe; sin datos sensibles ni credenciales reales. |

## 6. HALLAZGOS Y ACCIONES RECOMENDADAS

| N.° | Hallazgo | Acción recomendada | Responsable sugerido |
|---|---|---|---|
| H-01 | El criterio "dos solicitudes simultáneas no dejan stock negativo" no cuenta con una prueba de concurrencia real; solo está respaldado por el diseño del código (SELECT ... FOR UPDATE). | Agregar un caso de prueba que lance dos llamadas en paralelo a crear_pedido_transaccion sobre el mismo producto y verifique que el stock final sea correcto y no negativo. | Renzo |
| H-02 | No existe un caso de prueba que intente comprar una cantidad mayor al stock disponible contra la base de datos real (PGlite); solo se infiere leyendo el código SQL. | Añadir un test a database.test.js que compre más unidades de las disponibles y verifique el rechazo y que el stock no cambie. | Renzo / Brant |

| N.° | Hallazgo | Acción recomendada | Responsable sugerido |
|---|---|---|---|
| H-03 | Sin un archivo .env configurado, 3 de 10 suites del backend no cargan (mismo hallazgo que en el informe de Sprint 3). | Documentar variables de entorno de prueba (sin secretos reales) para CI, o versionar un .env.test de ejemplo. | Renzo |
| H-04 | El Sprint Backlog planificado no coincide con el estado real del repositorio: todo el código corresponde a un único commit que ya incluye funcionalidades de varios sprints, incluido RF-07. | Mantener commits incrementales por sprint en adelante para sustentar trazabilidad cronológica real. | Ambos |

## 7. CONCLUSIONES

- RF-07 tiene una base sólida: la función crear_pedido_transaccion bloquea la fila del producto, valida stock, modalidad de entrega y vencimiento, y crea el pedido junto con la reducción de stock dentro de una única transacción atómica.
- La clave de idempotencia, generada en el cliente y reutilizada en reintentos, evita pedidos duplicados: está probada con evidencia real contra una base de datos PostgreSQL en memoria (PGlite).
- De los ocho casos de prueba propuestos para este sprint, seis cuentan con evidencia real de ejecución (PASS) y dos permanecen pendientes: compra por encima del stock disponible, y compra concurrente sobre las mismas unidades.
- En conjunto, 140 pruebas automatizadas reales se ejecutaron entre backend y frontend sin fallos; la limitación de entorno (.env) que impedía cargar algunas suites de backend, señalada también en el informe de Sprint 3, se mantiene como pendiente.
- El objetivo central del Sprint 4 —reserva de stock atómica y consistente— se cumple en el código; para cerrar completamente el Definition of Done faltan dos pruebas por agregar, no cambios de funcionalidad.

## 8. BIBLIOGRAFÍA Y FUENTES TÉCNICAS


- Grupo G01. (2026). Planificación de Sprints de ÓrbiKa, versión 1.0, 12 de octubre de 2026: Product Backlog, Sprint Backlog y matriz de trazabilidad.
- Grupo G01. (2026). Informe de Trazabilidad — Sprint 3 ÓrbiKa (documento de referencia para el formato de este informe).
- Grupo G01. (2026). Especificación de requisitos de software (SRS), RF-07 y CU-05.
- Repositorio ORBIKA. (2026). supabase/migrations/002_security_and_order_transactions.sql y 003_srs_integrity_and_images.sql.
- Repositorio ORBIKA. (2026). backend/src/services/pedido.service.js, backend/src/validation/pedido.validation.js, backend/src/controllers/pedido.controller.js, backend/src/routes/pedido.routes.js.
- Repositorio ORBIKA. (2026). backend/test/sql/database.test.js, backend/test/services/pedido.service.test.js.
- Repositorio ORBIKA. (2026). frontend/src/pages/ProductDetail.jsx, frontend/src/pages/product.helpers.js, frontend/src/services/marketplace.js